FDS COURSE PROJECT · PGDBA BATCH 12 · IIM CALCUTTA

PlaceIQ

A campus placement & recruitment database that enforces its own rules

Fundamentals of Database Systems · Prof. Debapriyay Mukhopadhyay

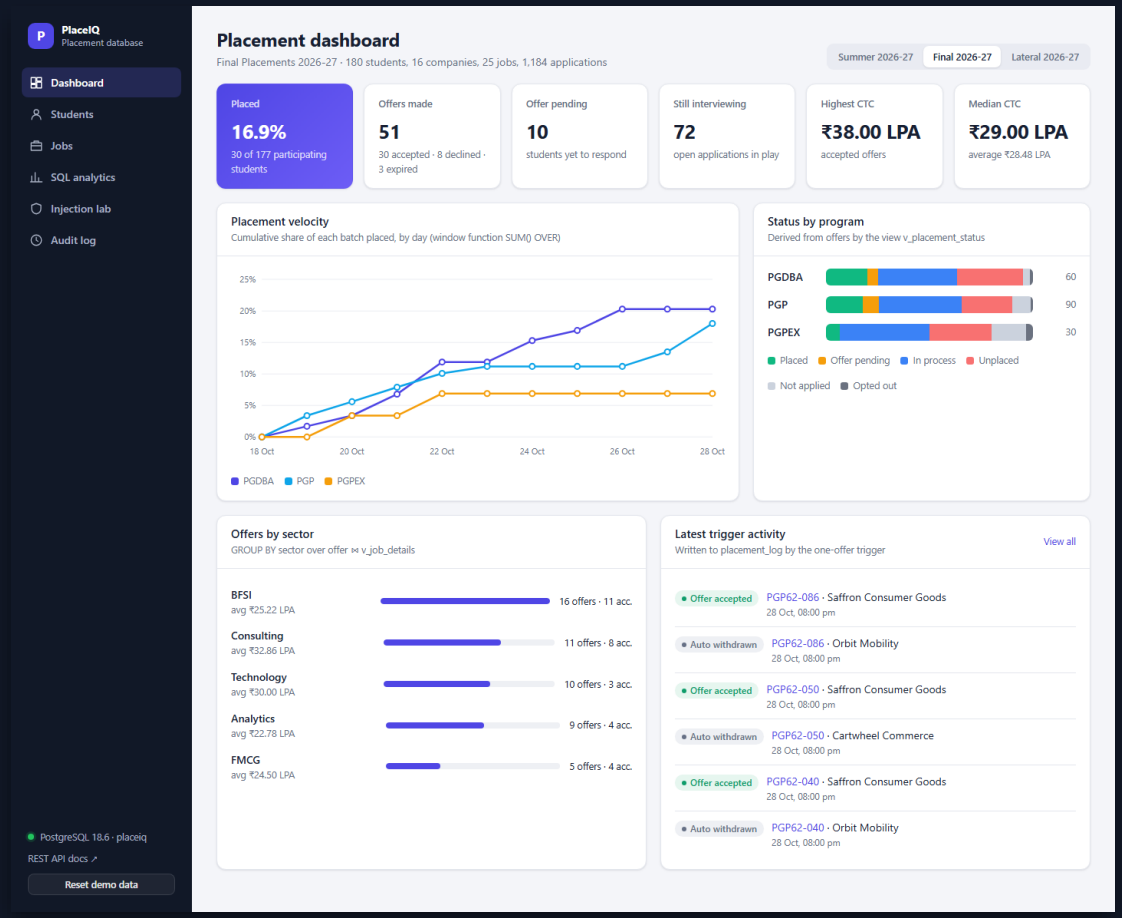
M1 Ekta Sharma

M2 [Member 2]

M3 [Member 3]

M4 [Member 4]

November 2026



Live web UI over PostgreSQL 18: every number on screen is computed by SQL

Four parts, four presenters

M1 Ekta Sharma

M1

Ekta Sharma

Problem & EER

0:00 – 3:15

- Problem & anomalies
- Business rules
- EER model: specialization, weak entity, aggregation, union type

M2

[Member 2]

Schema, normalization & DDL

3:15 – 7:00

- EER → relational mapping, keys
- FDs, 1NF → BCNF, lossless & dependency-preserving
- DDL: constraints as rules

M3

[Member 3]

SQL queries & rules

7:00 – 10:30

- Seeded, reproducible data
- Five queries: division, windows, HAVING
- Triggers & live demo

M4

[Member 4]

Analytics & performance

10:30 – 15:00

- Statistics in SQL
- Feature engineering, k-means
- Segments back in the DB; indexing, security

One spreadsheet, three anomalies

M1 Ekta Sharma

roll_no	company	hq_city	role	pay	round	result
DBA12-007	Northbridge	Mumbai	Assoc. Consultant	34 LPA	1 CASE	CLEARED
DBA12-007	Northbridge	Mumbai	Assoc. Consultant	34 LPA	2 CASE	SCHEDULED
DBA12-019	Northbridge	Mumbai	Assoc. Consultant	34 LPA	—	REJECTED
PGP62-003	Northbridge	Mumbai	Assoc. Consultant	34 LPA	1 CASE	CLEARED

Company, city, role and pay repeat on every applicant × round row

Update anomaly

Northbridge moves HQ: edit every row, miss one and the data contradicts itself.

Insertion anomaly

A new job cannot be recorded until a student applies (roll_no is part of the key).

Deletion anomaly

Delete the only applicant and the job's pay, cut-off and rounds disappear too.

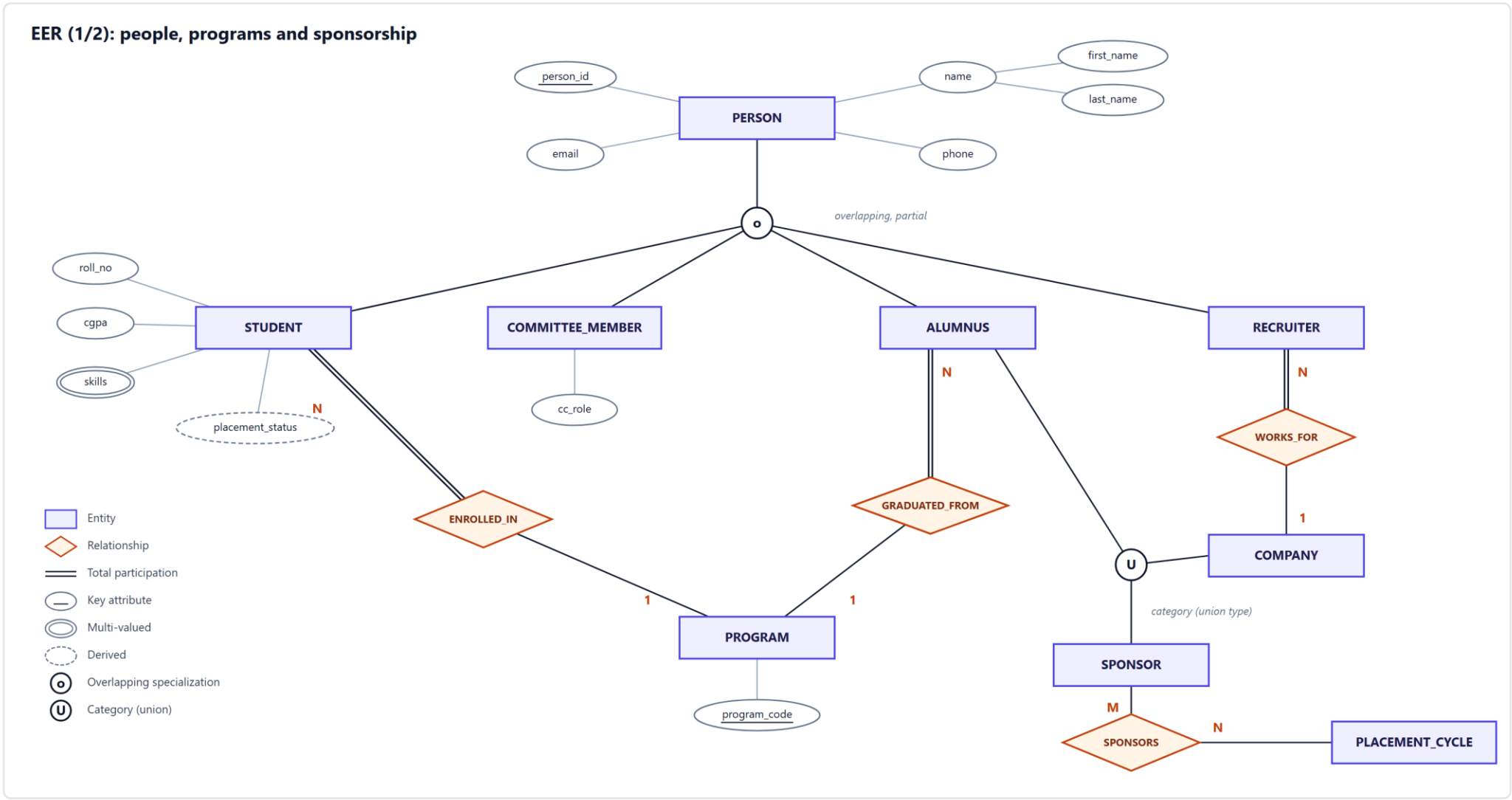
15

business rules, each enforced by the database, not by people

- Eligibility: program, CGPA cut-off, backlogs, deadline
- No overlapping interview slots
- One accepted offer per cycle, with automatic withdrawal elsewhere
- Placed status derived, never typed in
- A student holding an offer cannot be deleted

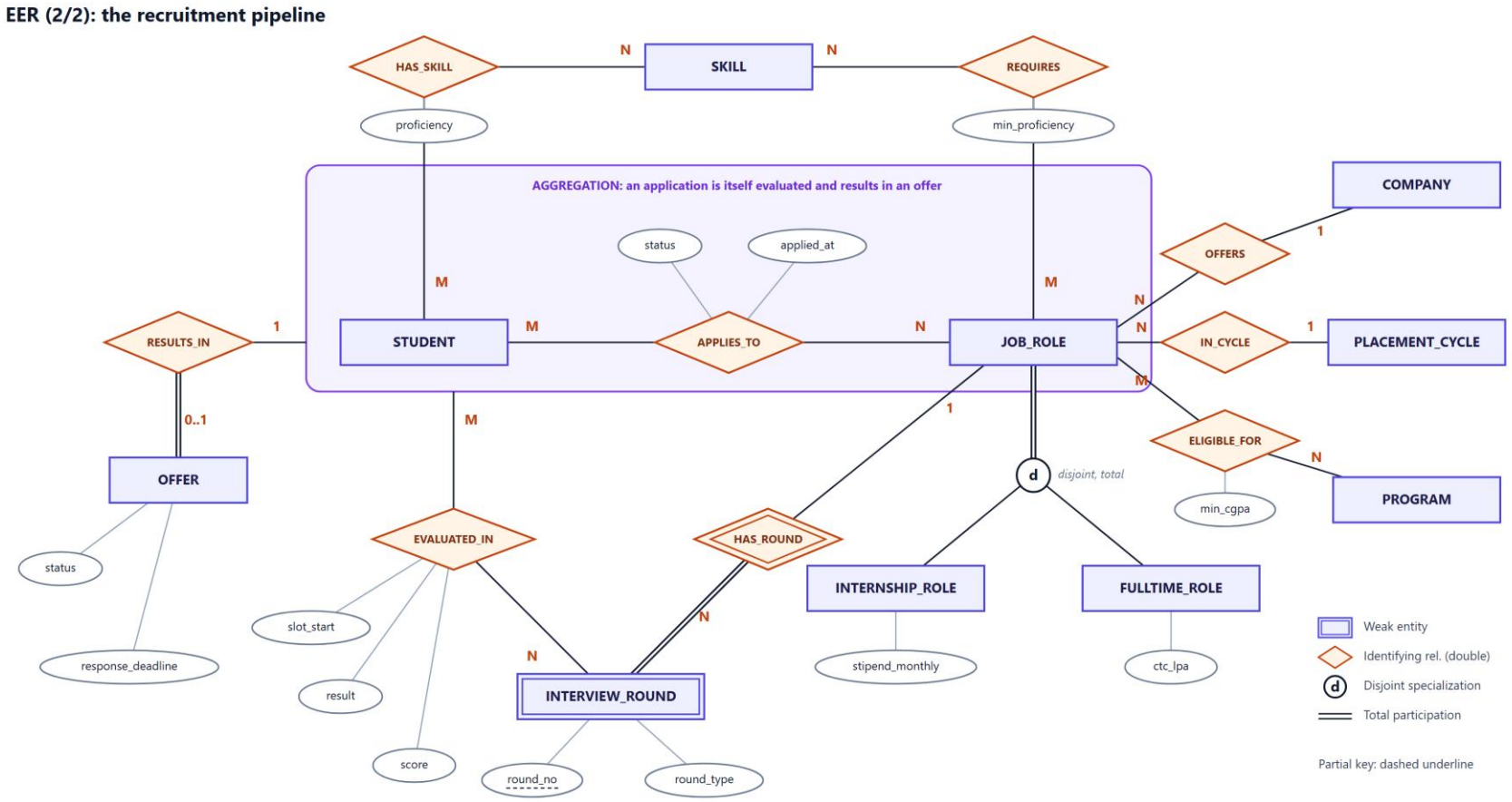
EER model (1/2): people, programs and sponsors

M1 Ekta Sharma



EER model (2/2): the recruitment pipeline

M1 Ekta Sharma



Aggregation

APPLIES_TO is treated as an entity: interviews and offers attach to an application, not to a student.

Weak entity

INTERVIEW_ROUND is identified by (job_id, round_no) through HAS_ROUND.

Disjoint & total

Every job is exactly one of internship or full-time ('d', double line).

From EER to 22 relations, and their keys

M2 [Member 2]

Step	Mapping rule	Applied to
1	Strong entity → table	program, person, company, job_role …
2	Weak entity → PK = owner PK + partial key	interview_round (job_id, round_no)
3–4	1:1 / 1:N → foreign key on the N side	student.program_id, offer → application
5	M:N → bridge table	application, eligibility, interview
6	Multi-valued attribute → own table	student_skill
8A	Specialization → table per subclass	student, recruiter; internship_role …
9	Category → surrogate key + FK in members	sponsor ← company, alumnus

Super key

Any attribute set that identifies a row: (person_id, email)

Candidate keys

Minimal super keys. placement_cycle has three: cycle_id, cycle_name, (cycle_type, academic_year)

Primary & alternate

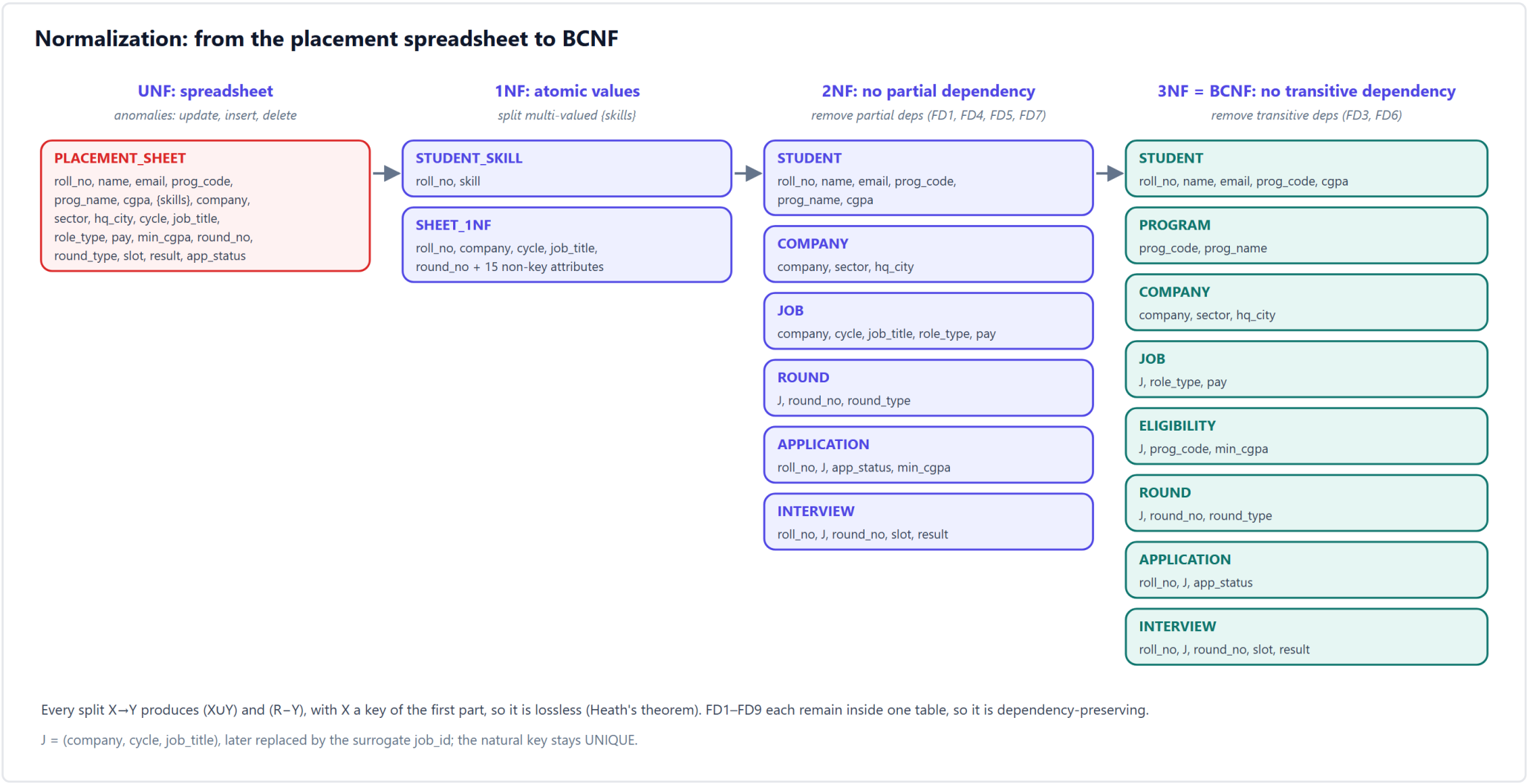
job_id is primary; (company_id, cycle_id, title) stays UNIQUE as the alternate key

Foreign keys

30 FKs, some composite: (posted_by, company_id) → recruiter

Normalization: spreadsheet → BCNF

M2 [Member 2]



Every split X→Y produces (XUY) and (R−Y), with X a key of the first part, so it is lossless (Heath's theorem). FD1–FD9 each remain inside one table, so it is dependency-preserving.

J = (company, cycle, job_title), later replaced by the surrogate job_id; the natural key stays UNIQUE.

Proving BCNF, and the trap we avoided

M2 [Member 2]

Relation	Non-trivial FDs	Determinants are keys?
student	roll_no → all; email → roll_no	Yes: both are candidate keys
program	prog_code → prog_name	Yes
eligibility	(job, program) → min_cgpa	Yes
interview_round	(job, round_no) → round_type	Yes
interview	(student, job, round) → slot, result	Yes
placement_cycle	3 candidate keys, each → all	Yes

All 22 relations: every determinant is a super key → BCNF

Deliberately not stored: CTC on the offer (offer → job → ctc would be transitive), placement status (derived by a view), cycle on the application (job → cycle).

The 3NF-but-not-BCNF trap

Suppose: each recruiter interviews for exactly one job.

Panel(student, job, recruiter) with (student, job) → recruiter and recruiter → job.

recruiter is not a super key → not BCNF. job is prime → still 3NF.

BCNF split: (recruiter, job) + (student, recruiter) is lossless but NOT dependency-preserving.

Our decision: the business does not need that rule, so we did not model it. The schema stays BCNF with nothing lost.

Integrity lives in the schema

M2 [Member 2]

```
-- Disjoint specialization, enforced declaratively
CREATE TABLE internship_role (
  job_id     INT PRIMARY KEY,
  role_type  VARCHAR(10) DEFAULT 'INTERNSHIP'
             CHECK (role_type = 'INTERNSHIP'),
  stipend_monthly NUMERIC(10,2) NOT NULL,
  FOREIGN KEY (job_id, role_type)
    REFERENCES job_role (job_id, role_type)
    ON DELETE CASCADE
);

-- No student in two interviews at once
CONSTRAINT excl_student_double_booking
  EXCLUDE USING gist (
    student_id WITH =,
    tstzrange(slot_start, slot_end) WITH &&)
```

45
CHECK constraints (domains)

30
foreign keys, with actions

17
UNIQUE (alternate keys)

1
EXCLUDE (overlap ban)

3
deferred constraint triggers

55
indexes incl. FK indexes

ON DELETE, chosen per relationship

CASCADE: subtypes & weak entity · RESTRICT: history (offers) · SET NULL (col): keep job, drop recruiter link

A realistic, reproducible test bed

M3 [Member 3]

180
students in 3 programs

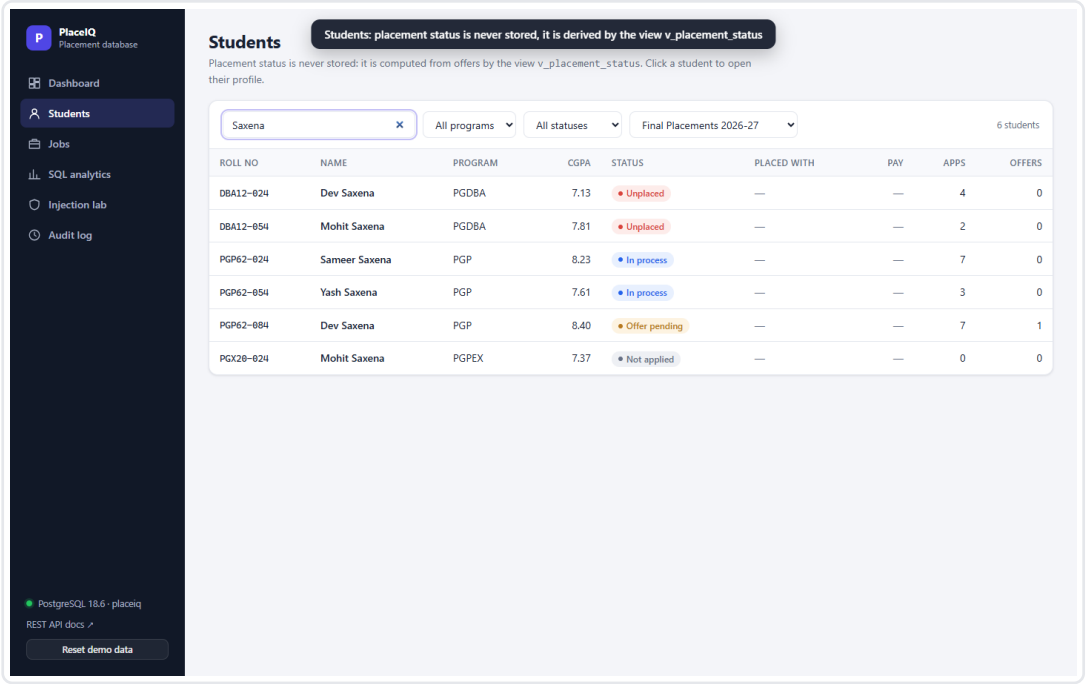
16
fictional companies

1,184
applications

1,057
interview slots

96
offers made

- Reference data is hand-written INSERTs, readable on a slide
- Students are generated set-based; setseed() makes every run identical
- A PL/pgSQL simulation runs shortlist → rounds → offers → accept, job by job
- Every row is loaded THROUGH the triggers, in one transaction: it all passed every rule, or nothing loads



Five queries, five business questions

M3 [Member 3]

#	Business question	SQL technique	Answer from our data
Q1	Which recruiters are hardest to crack?	Joins, outer join, GROUP BY, HAVING vs scalar subquery	5 companies; Meridian Insurance offers to 8.6%
Q2	Who holds EVERY skill a new job needs?	Relational division: double NOT EXISTS	8 students, led by DBA12-039
Q3	Top earners per program?	DENSE_RANK, NTILE, AVG() OVER on a view	Top CTC ₹38 LPA
Q4	How fast is each batch getting placed?	CTEs, generate_series, running SUM() OVER	20.3% of PGDBA placed by 28 Oct
Q5	Strong students lost in interviews?	Correlated subquery + LATERAL join	3 students flagged for coaching

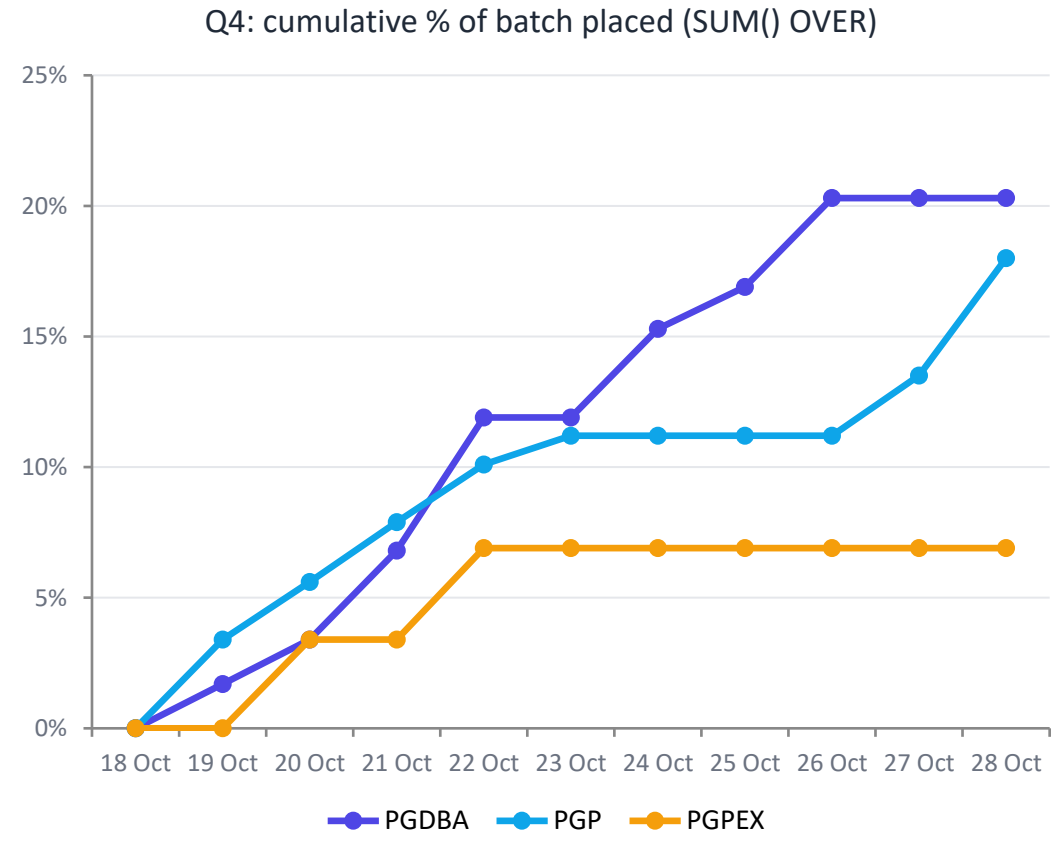
Spotlight: relational division and window functions

M3 [Member 3]

```
-- students holding EVERY skill job 25 needs
SELECT s.roll_no FROM student s
WHERE NOT EXISTS (          -- no required skill
  SELECT 1 FROM job_required_skill r
  WHERE r.job_id = 25
    AND NOT EXISTS (        -- ... that s lacks
      SELECT 1 FROM student_skill ss
      WHERE ss.student_id = s.person_id
        AND ss.skill_id   = r.skill_id
        AND ss.proficiency >= r.min_proficiency));
```

Qualified(student, skill) ÷ π skill (σ job = 25 (JobRequiredSkill))

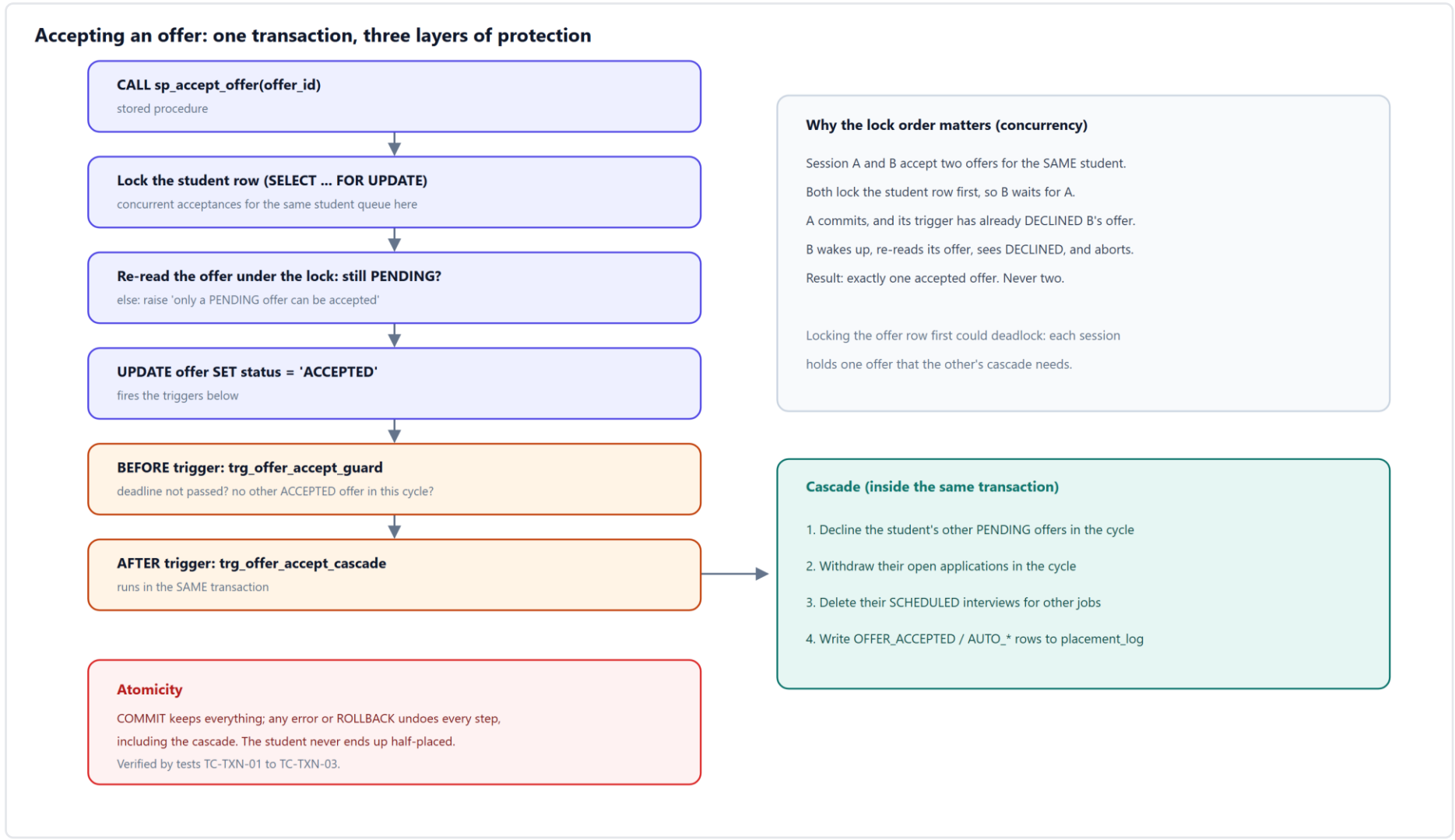
Cross-checked against division-by-counting (GROUP BY … HAVING count = …) for all 25 jobs: identical results (test TC-QRY-06).



Window functions aggregate without collapsing rows: each day keeps its own row and gets a running total.

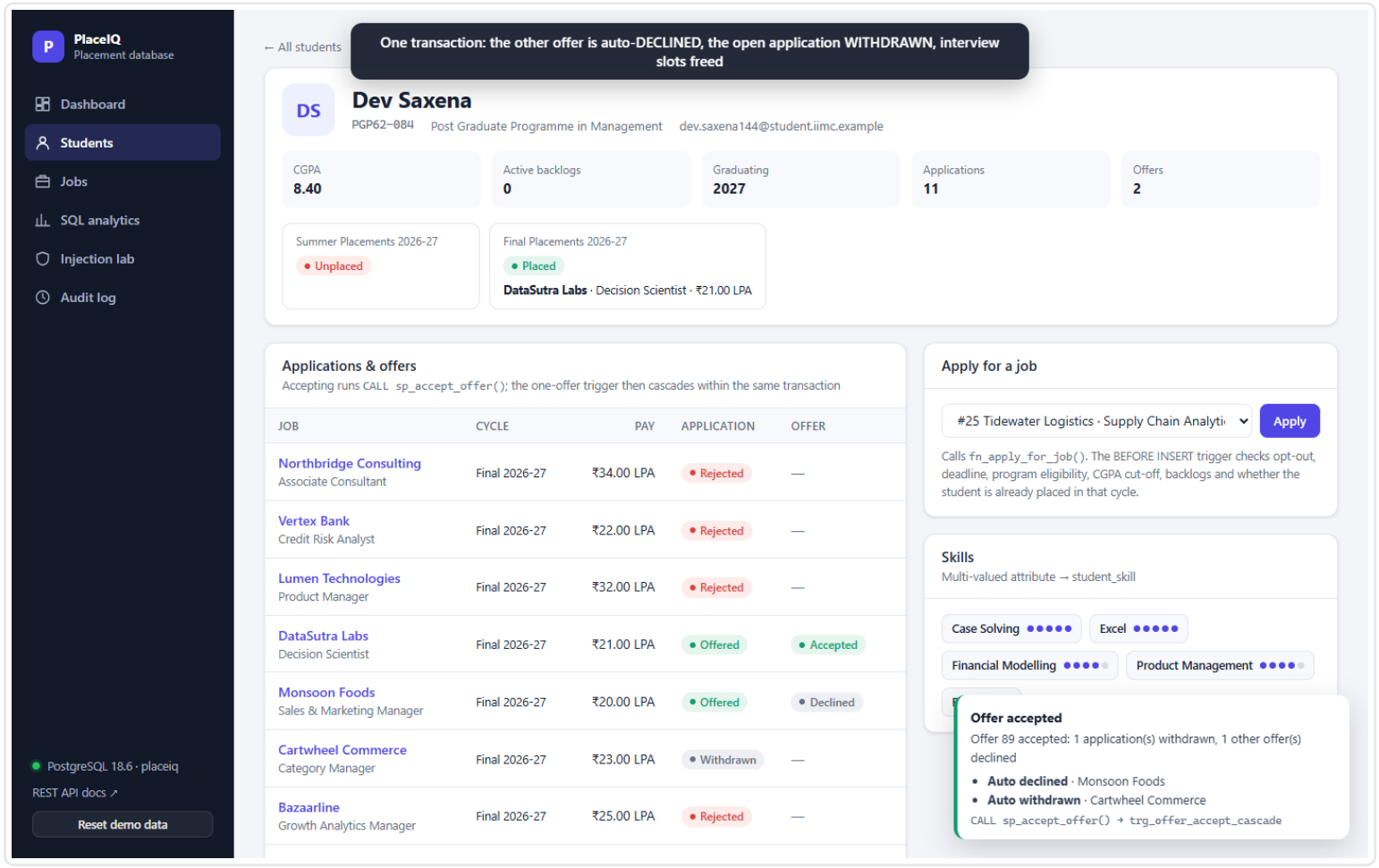
The one-offer policy, as a trigger

M3 [Member 3]



Live demo: accept an offer, watch the cascade

M3 [Member 3]



1. **Second offer**
   Monsoon Foods makes Dev Saxena an offer; trigger marks the application OFFERED

2. **Accept**
   Dev accepts DataSutra: CALL sp_accept_offer()

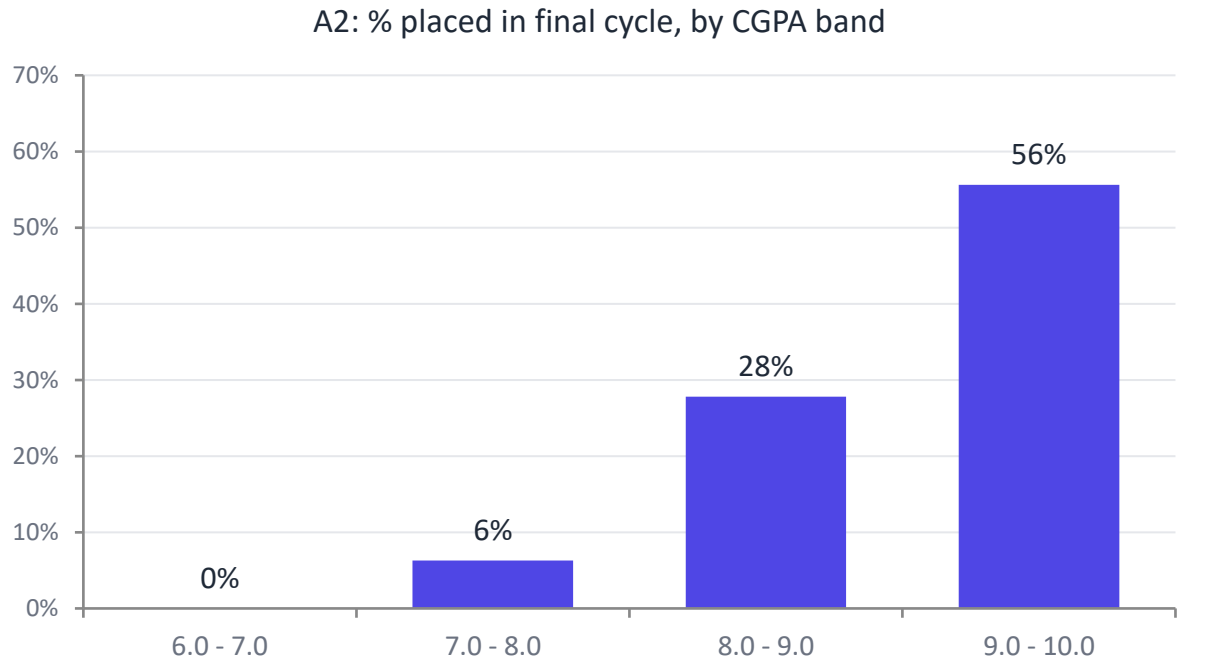
3. **Cascade**
   Other offer DECLINED, open application WITHDRAWN, interviews freed, all logged

4. **Guard**
   Applying again, or a CGPA below the cut-off, is refused with the trigger's message

Backup if the live demo fails: PlaceIQ_Demo_Video.webm

Analytics (1/4): the database does the statistics

M4 [Member 4]




```
SELECT width_bucket(cgpa, 6, 10, 4) AS band, count(*),
       count(*) FILTER (WHERE status = 'PLACED')
FROM student LEFT JOIN v_placement_status ... GROUP BY band;
```

A1 · CGPA

Mean 7.88, median 7.87, SD 0.78: symmetric. ANOVA across programs p = 0.907: the programs do not differ.

A3 · Funnel

45–49% of applications reach an interview, but only 4–9% become offers: students fail after the interview, not at the CV.

A4 · Skill gap

SQL has the most jobs per proficient student, then Excel and Statistics: run workshops there first.

γ band; count(*), count(σ status='PLACED') (Student ⋈ σ cycle='FINAL' (PlacementStatus))

Analytics (2/4): features in SQL, k from data

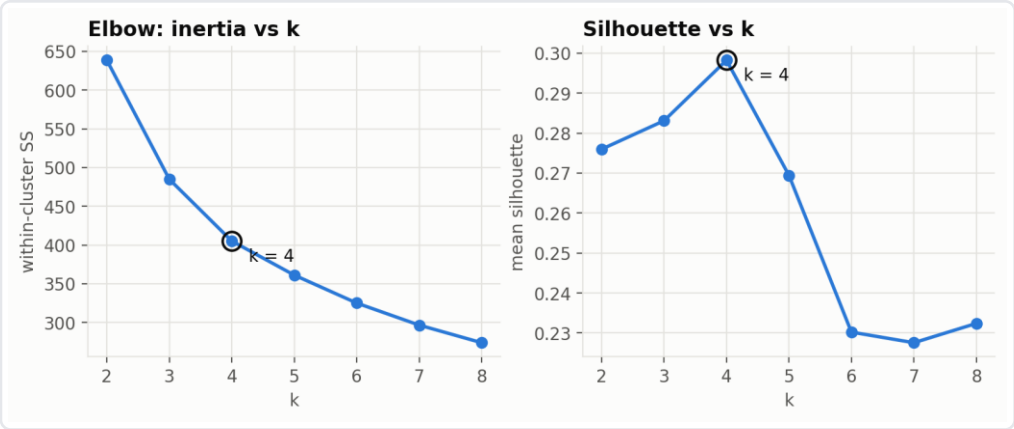
M4 [Member 4]

Feature	Why it should matter	Built from
cgpa	Recruiters' first filter (A2)	student
tech_score	Analytics roles screen on Python, SQL, ML	student_skill ⋈ skill
biz_score	Consulting & finance screen on cases, Excel	student_skill ⋈ skill
applications	No application, no offer	application
interview_rate	Share of applications reaching interview	application, interview

One view, one SELECT

v_student_features joins 6 normalized tables into 177 rows × 5 features: the only student-level data that leaves PostgreSQL.

Offers and CTC are deliberately NOT inputs: we cluster on what the committee sees early, then test the segments against outcomes. Otherwise the result would be circular.



Standardize first: k-means uses distance, so unscaled applications (0–14) would outvote interview_rate (0–1).

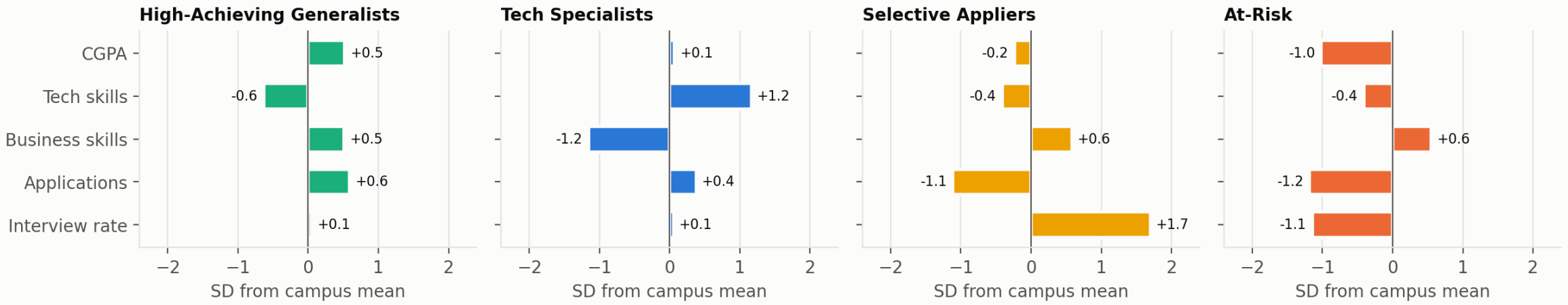
k = 4: inertia flattens after it and the silhouette peaks there (0.298).

Strongest structure: tech vs business skills, r = -0.68. Students specialise.

Analytics (3/4): four segments, profile only

M4 [Member 4]

Segment fingerprints: how far each centroid sits from the average student



High-Achieving Generalists

41% accepted · n = 68

High CGPA, business skills, apply widely

Tech Specialists

36% accepted · n = 56

Mostly PGDBA, technical skills, best CTC (₹29.3 L)

Selective Appliers

0% accepted · n = 19

Avg CGPA; 88% reach interview, but only 2.6 applications

At-Risk

3% accepted · n = 34

Low CGPA, few applications, rarely shortlisted

χ^2 test, segment × accepted offer: $\chi^2 = 25.7$, dof = 3, $p = 0.000011$. The segments differ in outcome, so the clusters are real, not just geometry.

Analytics (4/4): clusters back in the database

M4 [Member 4]

```
-- S2: who needs help now?
SELECT sg.segment_name, f.roll_no, ...,
  CASE WHEN f.applications < 3   THEN 'Apply to more roles'
       WHEN f.interview_rate < 0.4 THEN 'CV review' ... END
FROM student_segment g
JOIN segment sg USING (segment)
JOIN v_student_features f USING (student_id)
WHERE NOT f.accepted_offer
  AND g.segment IN (
    SELECT segment FROM v_segment_profile
    WHERE accepted_pct <
      (SELECT 100.0 * avg(accepted_offer::int)
       FROM v_student_features));
```

Weak = π segment (σ accepted_pct < campus avg (SegmentProfile)); result = σ ¬accepted (StudentSegment ⋉ Weak ⋈ Features)

Stored like any other data

Labels go into SEGMENT and STUDENT_SEGMENT (BCNF, foreign keys) in ONE transaction: every student is relabelled, or none is.

52 students flagged, each with one action

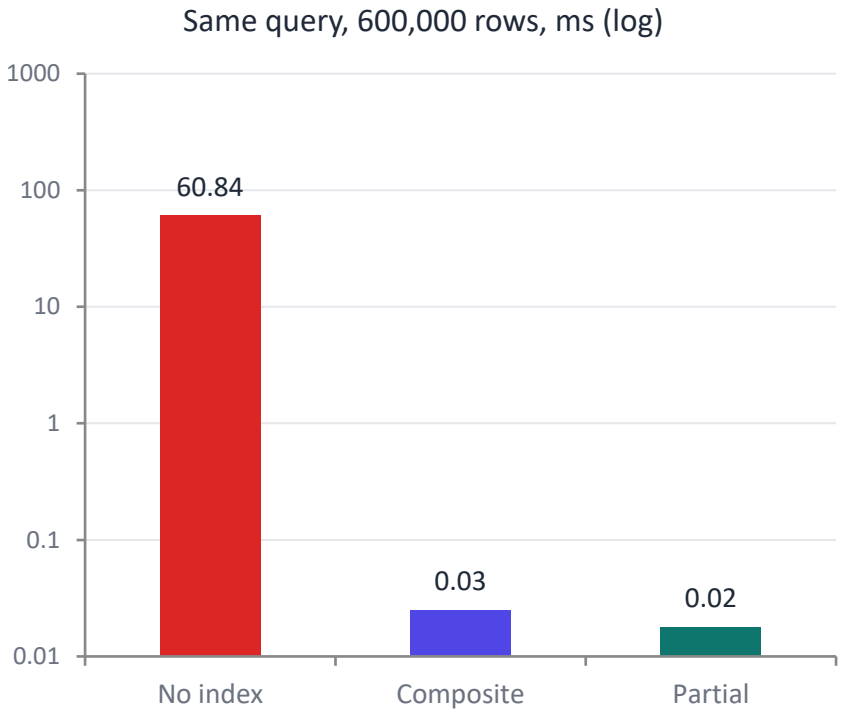
Segment	Suggested action	Students
At-Risk	Apply to more roles	19
At-Risk	CV review	12
At-Risk	Technical skills workshop	2
Selective Appliers	Apply to more roles	10
Selective Appliers	Technical skills workshop	8
Selective Appliers	Mock interviews	1

S1: PGP students fall into 4 segments and 52% of PGPEX are At-Risk, so a program-wide policy misses people.

Threshold t: flagging students whose P(offer) < 0.6 catches 89% of those with no offer, at 78% precision.

Indexing, security and testing

M4 [Member 4]



2,434×

faster: composite B-tree

26×

smaller: partial index

SQL injection: x' OR '1'='1

180

rows leaked by string concatenation

0

rows with a bound parameter

The unsafe path runs as a SELECT-only role in a READ ONLY transaction: DROP TABLE is refused.

91 / 91

automated tests pass against the real database: constraints, triggers, queries, concurrency, performance, security, API

What we learned

M1 Ekta Sharma

Modelling decides everything downstream: treating the application as an aggregation is what made interview and offer integrity enforceable.

M2 [Member 2]

Normalization is a proof: writing every FD exposed the transitive dependency hiding in min_cgpa. Declared constraints then hold even if the app is bypassed.

M3 [Member 3]

Rules that span tables need triggers and transactions: lock order turned a possible deadlock into a clean queue.

M4 [Member 4]

Cluster on profile, judge on outcome: the segments found Selective Appliers, whom a CGPA cut-off would never flag.

Thank you. Questions?

github.com/EktaSharma-github/placeiq · schema, data, queries, tests and the web UI